

Trabajo Práctico Nro 7

Fundamentos y Técnicas de Diseño de Pruebas

Materia: Metodología de los Sistemas I

Alumno: Sussini, Patricio

Institución: UTN – TUPaD

Parte 1: Fundamentos y Mentalidad del Tester

1. Testing de Software y el Costo Exponencial de un Defecto

El **Testing de Software** es el proceso planificado de evaluación y verificación de un producto de software con el objetivo de identificar defectos, confirmar que cumple con los requisitos especificados y garantizar su calidad antes de la entrega al usuario final. No se limita a ejecutar el programa y ver si funciona: incluye actividades de planificación, diseño de casos de prueba, ejecución y reporte de resultados.

La **metáfora del Costo Exponencial de un Defecto** establece que corregir un error se vuelve cada vez más costoso cuanto más tarde se lo detecta en el ciclo de vida del desarrollo. Un bug encontrado durante el relevamiento de requisitos es fácil de corregir porque solo implica modificar un documento o una definición. Ese mismo bug detectado ya en producción puede requerir revertir código en funcionamiento, comunicar a los usuarios afectados y reparar el daño causado, multiplicando el esfuerzo necesario varias veces.

¿Por qué es una inversión y no un gasto? Porque el costo de detectar un defecto temprano casi siempre es menor que el costo de corregirlo después. Invertir tiempo en testing evita problemas mayores más adelante: un bug crítico que llega a producción no solo cuesta horas de trabajo para corregirlo, sino también la confianza del cliente. En ese sentido, el testing protege tanto la calidad del producto como la reputación del equipo.

2. Mentalidad del Tester – Formulario de Login

La **mentalidad del tester** consiste en pensar de forma crítica y creativa frente a un sistema: buscar activamente los escenarios en los que podría fallar, en lugar de confirmar únicamente que el flujo esperado funciona. Mientras el desarrollador diseña pensando en el *happy path* (el camino exitoso), el tester piensa en los bordes, las excepciones y el comportamiento inesperado.

Tres preguntas que un tester haría sobre un formulario de login y que un desarrollador podría pasar por alto:

- **¿Qué pasa si el usuario ingresa 500 caracteres en el campo de contraseña?** El desarrollador probablemente probó con una contraseña de longitud normal. Una entrada excesivamente larga podría causar un error inesperado en la base de datos o romper la interfaz. El sistema debería limitar y validar la longitud máxima del campo.
- **¿Qué pasa si el usuario deja los dos campos en blanco y hace clic en 'Ingresar'?** Parece obvio, pero no siempre hay validación antes de enviar la solicitud. El tester verifica que el sistema muestre un mensaje de error claro en lugar de procesar una solicitud vacía.
- **¿Qué pasa si el usuario intenta iniciar sesión muchas veces seguidas con credenciales incorrectas?** El desarrollador implementó la lógica de autenticación pero pudo no haber pensado en este escenario. El tester verifica que el sistema tenga

algún mecanismo de protección, como bloquear la cuenta temporalmente o pedir una verificación adicional después de varios intentos fallidos.

3. Contraste de los tres enfoques de prueba

Pruebas de Caja Negra

Objetivo: Se prueba el sistema exclusivamente desde la perspectiva del usuario, sin conocimiento del código interno. El objetivo es verificar que las entradas producen las salidas esperadas según la especificación.

Ejemplo exclusivo: Verificar que el sistema aplica un descuento del 15% cuando el cliente es Premium y la compra supera \$50.000. Solo se ingresan los datos y se observa el resultado: no se necesita ni importa conocer el código interno.

Pruebas de Caja Blanca

Objetivo: Se prueba el código con conocimiento de su estructura interna, flujos y lógica. El objetivo es asegurar que todos los caminos de ejecución del código se ejerciten y que no existan ramas que nunca se ejecuten.

Ejemplo exclusivo: Verificar que el bloque ELSE de la función `es_mayor_de_edad()` se ejecuta al menos una vez durante las pruebas. Esta prueba es imposible de diseñar sin conocer la estructura interna del código.

Pruebas de Caja Gris

Objetivo: Combina elementos de los dos enfoques anteriores. El tester tiene conocimiento parcial del sistema (por ejemplo, conoce la estructura de la base de datos o cómo están organizados los módulos) y lo usa para diseñar pruebas más precisas, aunque siga probando desde afuera.

Ejemplo exclusivo: Un tester de Caja Gris que conoce la estructura de la base de datos puede verificar la cancelación de un turno de dos formas: desde la interfaz (que el turno no aparezca disponible) y también consultando directamente la tabla de turnos en la base de datos para confirmar que el registro cambió su estado a 'cancelado'. Esa segunda verificación, que requiere conocimiento interno, es lo que distingue a este enfoque de la Caja Negra pura.

Parte 2: Técnicas de Diseño de Pruebas de Caja Negra

1. Requisito 1 – Campo "Edad" (rango válido: 18 a 65 años, inclusive)

Particiones de Equivalencia identificadas:

Partición	Rango / Tipo	Valor representativo	Resultado esperado
PE1 (Válida)	$18 \leq \text{edad} \leq 65$	30	El sistema acepta el registro.
PE2 (Inválida)	$0 \leq \text{edad} < 18$	15	El sistema rechaza el registro (menor de edad).
PE3 (Inválida)	$\text{edad} > 65$	70	El sistema rechaza el registro (supera el límite).
PE4 (Inválida)	$\text{edad} < 0$	-5	El sistema rechaza el registro (valor imposible).
PE5 (Inválida)	Dato no numérico	"abc"	El sistema rechaza el registro (tipo de dato incorrecto).

Análisis de Valores Límite (BVA) – 6 casos de prueba:

Para el rango [18, 65] se analizan los valores en los bordes: inmediatamente por debajo del límite inferior, en el límite inferior, uno por encima; y uno por debajo del límite superior, en el límite superior y uno por encima:

# Caso	Valor de edad	Partición	Resultado esperado
BVA-01	17	PE2 (Inválida)	Rechaza – inmediatamente por debajo del límite inferior
BVA-02	18	PE1 (Válida)	Acepta – límite inferior exacto
BVA-03	19	PE1 (Válida)	Acepta – valor justo por encima del límite inferior
BVA-04	64	PE1 (Válida)	Acepta – valor justo por debajo del límite superior
BVA-05	65	PE1 (Válida)	Acepta – límite superior exacto
BVA-06	66	PE3 (Inválida)	Rechaza – inmediatamente por encima del límite superior

2. Requisito 2 – Tabla de Decisión: Descuento del 15%

El descuento del 15% se aplica **solo si** el cliente es **Premium Y** la compra es **mayor a \$50.000**. Con dos condiciones binarias se generan exactamente 4 combinaciones posibles ($2^2 = 4$):

	Regla 1	Regla 2	Regla 3	Regla 4
C1: ¿El cliente es Premium?	Sí	Sí	No	No
C2: ¿Compra > \$50.000?	Sí	No	Sí	No
ACCIÓN: Aplica descuento 15%	SÍ	NO	NO	NO

¿Por qué probar solo la Regla 1 ("Premium Y Compra > \$50.000") es insuficiente?

Probar únicamente el caso donde ambas condiciones son verdaderas solo verifica que el descuento se aplica cuando corresponde. Pero no verifica que el sistema **no** lo aplique cuando no debería. Si hubiera un error en la implementación de la condición, como aplicar el descuento cuando solo una de las dos condiciones se cumple, ese error pasaría completamente desapercibido. La Tabla de Decisión obliga a probar todas las

combinaciones, incluidos los casos en que el descuento NO debe aplicarse, que son igualmente críticos para el negocio.

Parte 3: Pruebas Estructurales y Niveles de Prueba

1. Cobertura de Código (Code Coverage)

La **Cobertura de Código** es una métrica que indica qué porcentaje del código fuente fue efectivamente ejecutado durante la ejecución del conjunto de pruebas. Sirve para detectar partes del código que nunca fueron probadas y que podrían esconder defectos. Los dos tipos más usados son:

- **Cobertura de Sentencias:** indica qué porcentaje de líneas de código se ejecutaron durante las pruebas. Es el tipo más básico.
- **Cobertura de Decisión (o de Ramas):** indica si cada rama condicional (el resultado verdadero y el falso de cada IF) fue ejercitada al menos una vez. Es más completa que la cobertura de sentencias porque obliga a probar todos los caminos posibles de una condición.

Una cobertura alta no garantiza que el software no tenga errores, pero una cobertura baja sí indica que hay partes del sistema que no fueron probadas.

2. Análisis de cobertura – función `es_mayor_de_edad()`

Pseudocódigo analizado:

```
Funcion es_mayor_de_edad(edad):  
IF edad >= 18:  
  Imprimir("Es mayor")  
ELSE:  
  Imprimir("Es menor")
```

¿Por qué una prueba con `edad = 25` es insuficiente? Con `edad = 25`, la condición "`edad >= 18`" es verdadera, por lo que el flujo entra en el bloque IF e imprime "Es mayor". El bloque ELSE nunca se ejecuta. Desde el punto de vista del usuario, el programa "funciona", pero existe una rama entera del código que no fue probada y que podría contener un defecto. La **Cobertura de Decisión alcanzada con esa única prueba es del 50%** (1 de 2 ramas posibles cubiertas).

¿Qué dos valores se necesitan para alcanzar el 100% de Cobertura de Decisión?

- **edad = 25** → la condición evalúa a VERDADERO → se ejecuta la rama IF.
- **edad = 17** → la condición evalúa a FALSO → se ejecuta la rama ELSE.

Con estas dos pruebas se cubren el 100% de las ramas de decisión de la función.

3. Pirámide del Testing

La **Pirámide del Testing** es un modelo muy conocido que se usa para explicar cómo distribuir los distintos tipos de pruebas en un proyecto. La idea central es que conviene

tener muchas pruebas unitarias (son rápidas y baratas de ejecutar), una cantidad razonable de pruebas de integración, y pocas pruebas de sistema completo, que son las más lentas y costosas de mantener.

- **Pruebas Unitarias (base):** verifican el comportamiento de la unidad mínima de código: una función, un método o una clase, de forma aislada. Son rápidas de ejecutar y muy precisas para ubicar el origen de un error. Ejemplo: probar que la función `calcularDescuento()` devuelve 15% cuando se le pasan los parámetros correctos.

- **Pruebas de Integración (nivel medio):** verifican que dos o más componentes del sistema funcionan correctamente cuando se combinan. Su objetivo es detectar problemas en cómo se comunican los módulos entre sí. Ejemplo: verificar que el módulo de autenticación se comunica correctamente con la base de datos y devuelve la respuesta esperada.

- **Pruebas de Sistema (cima):** verifican el sistema completo desde la perspectiva del usuario final, simulando flujos reales de uso. Son las más costosas y lentas, pero las más representativas del comportamiento real del producto. Ejemplo: simular el flujo completo de un paciente que solicita un turno médico, desde el login hasta la confirmación.

4. Pruebas Funcionales vs. Pruebas No Funcionales

Pruebas Funcionales: verifican que el sistema hace lo que debe hacer según los requisitos. Se centran en el **qué** hace el sistema, evaluando entradas y salidas concretas. Ejemplo: verificar que cuando un paciente selecciona un médico, elige un horario disponible y confirma el turno, el sistema registra el turno correctamente y envía la notificación de confirmación.

Pruebas No Funcionales: verifican **cómo** el sistema hace lo que hace. No validan un comportamiento funcional específico, sino atributos de calidad como el rendimiento, la seguridad o la usabilidad. Ejemplo: verificar que el sistema responde en un tiempo razonable cuando hay muchos usuarios conectados al mismo tiempo solicitando turnos.

Parte 4: Aplicaciones en el Mundo Real y Preguntas de Investigación

1. Bug famoso con impacto en el mundo real: Apagón global de Facebook (2021)

En octubre de 2021, Facebook, Instagram y WhatsApp estuvieron completamente caídos durante aproximadamente seis horas, dejando sin servicio a miles de millones de usuarios en todo el mundo. La causa fue un error en un cambio de configuración de la red interna de Facebook: al modificar las rutas de la red, los servidores quedaron desconectados entre sí y dejaron de ser accesibles desde Internet. Lo que agravó el problema fue que los propios sistemas internos de monitoreo y gestión también quedaron inaccesibles, lo que dificultó la recuperación.

El cambio que causó el fallo fue aplicado directamente sobre la infraestructura en producción sin haber sido validado previamente en un entorno separado que simulara su impacto sobre el resto del sistema.

Nivel de prueba que falló o fue insuficiente: en este caso es difícil hablar de un nivel de prueba de software tradicional, ya que el error no estaba en el código de la aplicación sino en el proceso de aplicar un cambio de configuración de red en producción. Lo que falló fue la validación previa al despliegue: no se probó el impacto del cambio en un entorno controlado antes de aplicarlo. Esto se relaciona más con las Pruebas de Sistema y con los procesos de gestión de cambios que con una prueba de integración de código tradicional. El caso ilustra que los fallos en sistemas reales no siempre provienen de errores en el código, sino también de decisiones en el proceso de despliegue.

Fuente: Blog de ingeniería de Meta. "More details about the October 4 outage." [engineering.fb.com](https://engineering.fb.com/2021/10/04/outage/), octubre 2021.

2. Preguntas de Investigación

Pregunta 1: Selenium vs. Cypress – Automatización de pruebas de UI

Selenium es una herramienta de código abierto para automatizar pruebas sobre navegadores web, disponible desde 2004. Funciona enviando instrucciones al navegador desde afuera, a través de un componente llamado driver (uno distinto para cada navegador). Su principal ventaja es que soporta múltiples lenguajes de programación para escribir los tests (Java, Python, JavaScript, entre otros) y permite probar en diferentes navegadores, lo que lo hace muy flexible para equipos con tecnologías variadas.

Cypress es una herramienta más moderna, lanzada en 2017, pensada exclusivamente para aplicaciones web. A diferencia de Selenium, se ejecuta dentro del mismo navegador en el que corre la aplicación, lo que la hace más rápida y estable en muchos escenarios, especialmente en aplicaciones web modernas. Su configuración es más simple y ofrece mejores herramientas para ver qué pasó durante las pruebas. Su limitación principal es

que los tests deben escribirse en JavaScript o TypeScript, lo que puede ser una restricción para equipos que no trabajen con esos lenguajes.

¿Cuándo usar uno sobre el otro?

- Usar **Selenium** cuando el proyecto necesita probarse en múltiples navegadores, o cuando el equipo prefiere escribir los tests en un lenguaje distinto a JavaScript.
- Usar **Cypress** cuando se prioriza la facilidad de configuración y la velocidad de ejecución de las pruebas, y el equipo trabaja con JavaScript o TypeScript.

Fuentes: docs.cypress.io; selenium.dev

Pregunta 2: Testing de Regresión y su rol en las metodologías ágiles

El **Testing de Regresión** consiste en volver a ejecutar pruebas ya existentes cada vez que se realiza un cambio en el sistema (una nueva funcionalidad, la corrección de un bug, una refactorización) para verificar que el cambio no rompió algo que antes funcionaba correctamente. En otras palabras, protege lo que ya estaba validado frente a las modificaciones continuas del código.

¿Por qué es un candidato ideal para la automatización en metodologías ágiles? En Scrum, el código cambia en cada Sprint. Ejecutar manualmente todas las pruebas existentes al final de cada iteración consumiría demasiado tiempo. Si ese conjunto de pruebas está automatizado, se puede ejecutar rápidamente cada vez que el equipo hace un cambio, detectando problemas de forma inmediata y sin depender de disponibilidad humana. Esto permite al equipo mantener la velocidad de entrega sin sacrificar la estabilidad del producto.

Fuentes: ISTQB Foundation Level Syllabus – Pruebas de Regresión; material de cátedra Unidad 7.